Power User's Guide to

ProDG for Nintendo GameCube™ Build Tools

SN Systems

SN Systems Ltd
Version 3.9.3
May 2003

Copyright © SN Systems Ltd, 2000-2003. All rights reserved.

"ProDG" and the SN logo are registered trademarks of SN Systems Ltd. "GameCube" is a trademark of Nintendo Corp. "Microsoft", "MS-DOS", "Visual Studio", "Win32", "Windows" and "Windows NT" are registered trademarks of Microsoft Corporation. "GNU" is a trademark of the Free Software Foundation. Other product and company names mentioned herein may be the trademarks of their respective owners.

NINTENDO GAMECUBE™

NINTENDO GAMECUBE は、任天堂株式会社の登録商標です。
Windows、WindowsNTは米国Microsoft Corporationの米国およびその他の国における登録商標または商標です。製品および内容物の外見、形状、仕様は、予告なしに変更される場合があります。

ProDGNGC_PU(BT)-E / Power User Guide to ProDG for Nintendo GameCube Build Tools / May 2003

Document change control

Ver.	Date	Changes
3.9.3	May 2003	Released with ProDG for Nintendo GameCube v3.9.3. Changes since v3.9.2: added info on source browsing and viewing dependencies in VSI chapter; added several 'Troubleshooting' sections; new chapter 'Optimizing your builds'; split out info on 'Relocatable DLLs' to new chapter; new 'Utilities' chapter with info on blankelf and make; unsupported assembler/linker options and error messages relegated to appendices; amended table of compiler C/C++ switches and warnings; corrected format for assembler error and warning messages; corrected -x<type> ngccc switch; #pragma mathwarn added; new 'Additional compiler features' section; new ngcld switches: -sn-full-map, -sn-overlays and -whole-archive; added setup steps for VSI chapter; new snarl MRI scripting documentation; updated optimization chapter; corrected 'asm volatile' statements.

Contents

Chapter 1: Introduction 1

Overview 1
Upgrading to ProDG Plus 1
Technical support and upgrades 2
Updating ProDG over the web 2
Updating the Nintendo SDK 3

Chapter 2: Using the command-line build tools 5

ProDG build tools 5
ProDG utilities 5
GNU build tools 6
sn.ini - SN Systems configuration file 6
Using variables in sn.ini 7
libsn.a - SN Systems library 8
Obtaining the version for libsn.a 8

Chapter 3: The Visual Studio 6.0 integration 9

Overview of the Visual Studio 6.0 integration 9
Setting up Visual Studio 9
Creating a GameCube project in Visual Studio 9
The active project configuration 11
Configuring the Visual Studio 6.0 integration 12
Building your GameCube project 13
Adding your project files 13
Building your project 14
Loading and running ELF files 15
Integration with the ProDG Debugger 15
Setting breakpoints in your project 16
Editing your source in Visual Studio 16
Removing temporary files 16
Generating dependency information 17
How Visual C++ generates dependency information 17
Source browsing 17
How Visual C++ generates browse information 17
Improving browse information generation using the NGC_in_VC.h file ... 18
Improving browse information generation by modifying source code 18
Using source browsing with makefiles 19
Troubleshooting 19
Installing breaks other third-party integrations 19
'Reference to undefined symbol' error 20

Chapter 4: Memory management in C and C++ 21

Introduction 21
C++ Exception handling malloc/free missing 21
strdup (malloc only) 21
setvbuf (malloc/free) 21
Direct call to malloc 22
Direct call to free 22
Programming with malloc and free 22
C++ programming with new and delete 23
Using I/O stream buffers 24

Chapter 5: The ProDG compiler driver 27

Using the ngccc compiler driver 27
ngccc command-line syntax 28
ngccc options 28
Filenames 32
Overriding default file extension actions 34
Specifying the GameCube libraries 34
Using a response file 34
C programming considerations 35
Global variables at fixed memory locations 35
New 'sync' intrinsic 35
Naked function support 35
Fast casting 36
Implementation 36
Programming considerations 36
Paired single floating-point operations 37
Type implementation 37
Conversions 38
Native PS operations 39
SN API for PS (SNAPS) Operations 39
General programming considerations 41
Example 42
GameCube programming tips 45
Troubleshooting 46
C compiler not giving some warnings 46
'Unknown escape sequence' error 47

Chapter 6: The ProDG assembler 49

Using the SN Systems assembler 49
Command-line syntax 49
Assembler directives 50
SN Systems directives 51
Naming registers and register fields 52
Scope delimiting 52
Address modes 53
Input files 53
Filenames and line numbers 53
Output (object) file 54

Error and warning messages 54
Symbol names 55
Local symbols 55
Register referencing 55
Register numbers 56
Register names 56
Register names and inline assembler parameters 56
Registers and input/output/clobber lists 56
Debugging your inline assembler 58

Chapter 7: The ProDG linker 59

Using the ngcld linker 59
Memory requirements 59
Linker scripts 59
Sections 60
Referencing files in linker scripts 60
ngcld command-line syntax 61
ngcld switches 61
Small data area support 63
Why use the small data area? 63
Increasing or decreasing the use of small data sections 63
Troubleshooting 64
'Reference to undefined symbol' error 64

Chapter 8: Optimizing your builds 65

Introduction 65
Optimizing for small ELF size 65
Compiler options 65
Linker options 66
Removing symbol information 67
Removing unnecessary debug information 67
Preventing generation of non-inlined function copies 67
Removing global destructors 67
Optimizing for fast execution 68
Using the compiler optimization switches 68
Optimizing for build speed 68
Avoid unnecessary include files 69
Maximize system resources 69
Technical support 70
Feedback to SN Systems 70

Chapter 9: Relocatable modules 71

Building a relocatable module 71

Chapter 10: Utilities 75

blankelf - blank ELF utility 75
make 75
snarl - SN Archive Librarian 76

Library reading 76
Cross-platform 76
Command-line syntax 76
Verbose mode 78
Print archive files 78
Printing object files 78
Displaying the symbol table 78
Super fast append 78
Warning of multiply-defined symbols 79
Symbol manipulation commands 80
Response file scripting 80
MRI scripting 80
Running MRI scripts from files 83

Chapter 11: Building for Gekko profiling **85**

How to support Gekko profiling in your game 85
Gekko profiler API 85

Appendix A: Assembler reference **89**

Unsupported assembler options 89
Unsupported assembler directives 89

Appendix B: Linker reference **91**

Unsupported ld switches 91
Unsupported script file directives in ngcld 92
Error messages 93
Generic error messages 93
ngcld specific error messages 95

Index **97**

Chapter 1: Introduction

Overview

ProDG is SN Systems' suite of fully featured Win32 tools that enable you to build and debug your games for Nintendo's GameCube console. Using the ProDG tools you can now build your application on a Win32 PC and upload the results to the GameCube development hardware for running and debugging.

ProDG consists of:

- **ProDG Build Tools**: Microsoft Visual Studio integration option; ProDG compiler driver; ProDG linker; SN Systems utilities and library; ProDG assemblers.
- **ProDG Target Manager** that enables you to control connection to the GameCube targets in your network. See *User Guide to ProDG Target Manager for Nintendo GameCube.*
- **ProDG Debugger**: a fully featured Win32 debugger for debugging your GameCube applications. The Debugger is described in *Power User's Guide to ProDG for Nintendo GameCube Debugger.*

This manual covers the ProDG Build Tools.

Upgrading to ProDG Plus

ProDG Plus is a suite of advanced game development and debugging tools for the GameCube.

- **ProDG** — a suite of development tools for building and debugging GameCube games. It consists of a C/C++ compiler, assemblers, linker, debugger and target manager. An optional Visual Studio Integration provides App-Wizards for building executables and libraries, and launching the Debugger from within Visual Studio.
- **Advanced debugger features** — debugger scripting, and other enhanced features to be announced, provide ProDG Plus customers with the 'gold standard' in GameCube debuggers.
- **Tuner** — lets you capture and visualize program behavior so that you can eliminate conflicts and bottlenecks in your code. High performance games can now be achieved with less guesswork. Tuner captures data to a host PC in real-time while you play the

game. The captured data can then be analyzed frame by frame and saved for later comparison with your optimized code.

For details about upgrading from ProDG to ProDG Plus, please contact **sales@snsys.com**.

Technical support and upgrades

First line support for all SN Systems products is provided by the Support areas of our website. To view these pages you must be a registered user with an SN Systems User ID and Password.

- If you have forgotten your User ID and Password, send an e-mail to **webmaster@snsys.com**.

www.snsys.com/support (English)
www.snsys.jp/support (Japanese)

If the answer to your problem cannot be found on the Support areas of our website, you can also e-mail our support team at:

support@snsys.com (English)
j-support@snsys.com (Japanese)

Please make sure that you explain your problem clearly and include details of your software version and hardware setup. If you have been given an SN Systems support log number (LN number) then this should be quoted in all correspondence about the problem.

SN Systems Limited
4th Floor - Redcliff Quay
120 Redcliff Street
Bristol BS1 6HU
United Kingdom

Tel.: +44 (0)117 929 9733
Fax: +44 (0)117 929 9251

WWW: **www.snsys.com** (English)
www.snsys.jp (Japanese)

Updating ProDG over the web

Regular updates to ProDG are posted on the SN Systems web site (see "Technical support and upgrades" on page 2). You need to select **Technical Support** > **Downloads**, to access the Downloads area. You will be required to give your SN Systems User ID and Password to access these pages.

- Full product updates are provided in the **Full Install** section.

- Fully tested component updates are found in the **Product Updates** section.
- Beta versions of components may be provided in the **Pre-Release Files** section.

Updating the Nintendo SDK

Approved GameCube developers obtain Nintendo SDK updates directly from Nintendo. You do NOT normally need to reinstall ProDG for a minor upgrade to the Nintendo SDK.

- When Nintendo provides a major new release of the Nintendo SDK, DO NOT install it until you obtain a compatible build of ProDG from SN Systems. Information about the currently supported version of the Nintendo SDK is available from the SN Systems web site.

[THIS PAGE IS LEFT BLANK INTENTIONALLY]

Chapter 2: Using the command-line build tools

ProDG build tools

The installer creates the following directory tree:

.\ngc	Readme, ProDG settings and version utilities
.\ngc\bin	Executables
.\ngc\include	Standard C library header files
.\ngc\include\machine	Standard C library header files
.\ngc\include\sys	Standard C library header files
.\ngc\lib	Linkable object libraries
.\ngc\sndemo	SN Systems demos
.\ngc\stl	Standard template library

The following table details the ProDG build tools available for the Nintendo GameCube, with a brief description of what each does.

Command	Description
ngccc	Nintendo GameCube compiler driver. See "Using the ngccc compiler driver" on page 27.
ngcas	Nintendo GameCube assemblers. See "Using the SN Systems assembler" on page 49.
ngcld	ProDG Nintendo for GameCube linker. See "Using the ngcld linker" on page 59.

ProDG utilities

The following ProDG utilities are also available:

blankelf	Blank ELF utility. See "blankelf - blank ELF utility" on page 75.
make	A tool which controls the generation of executables and other files of a program.; make takes its input from a command file, with default name makefile. See "make" on page 75.

snarl	SN Archive Librarian. See "snarl - SN Archive Librarian.

GNU build tools

ProDG also ships with GNU C and C++ compilers and GameCube variants of a number of GNU utilities.

The following table details the GNU build tools available for the Nintendo GameCube, with a brief description of what each does. For detailed information on using individual GNU tools, refer to the online GNU documentation at **www.gnu.org**.

Command	Description
cc1	GNU C compiler. Normally called by a compiler driver such as gcc or ngccc. Files sent to cc1 should be preprocessed with cpp first.
cc1plus	GNU C++ compiler. Normally called by a compiler driver such as gcc or ngccc. Files sent to cc1plus should be preprocessed with cpp first.
cpp	GNU compiler pre-processor. Normally called by a compiler driver such as gcc or ngccc. Files sent to cc1 and cc1plus should be preprocessed with cpp first.

sn.ini - SN Systems configuration file

ProDG for Nintendo GameCube installs a configuration file called sn.ini, which is placed in a directory pointed to by the SN_NGC_PATH environment variable. The sn.ini file is used to specify which ProDG for Nintendo GameCube components you are using, particularly for the benefit of the Visual Studio integration and the ngccc compiler driver.

These are the default contents of sn.ini:

```
[ngccc]
compiler_path=<install dir>\bin
c_include_path=<install dir>\include
library_path=<install dir>\lib
assembler_path=<install dir>\bin
linker_path=<install dir>\bin
cplus_include path=<install dir>\include
linker_name=ngcld.exe
opt_assembler_name=ngcas
assembler_name=ngcas
linker_script=<install dir>\bin\ngc.ld
stdlib=<standard library list>
rel_link_file=<install dir>\bin\plfkeep.lst
rel_linker_script=<install dir>\bin\plf.ld
```

Lines starting with a '#' are treated as comments and are ignored, so you can easily swap between different versions of the build tools, assemblers

and linker by changing which lines of sn.ini are preceded by the '#' symbol.

Using variables in sn.ini

Variables can be used in the sn.ini file. These operate in the same way as internal variables in makefiles. For example:

```
[ngccc]
DOLPHIN_INC=c:\DolphinSDK1.0\include
DOLPHIN_LIB=c:\DolphinSDK1.0\HW2\lib
PRODG=c:\ngc

#compiler_path=$(PRODG)\bin\gcc3
compiler_path=$(PRODG)\bin

c_include_path=$(PRODG)\include;$(DOLPHIN_INC);$(PRODG);$(DOLPHIN_INC)
\dolphin
library_path=$(PRODG)\lib
assembler_path=$(PRODG)\bin
linker_path=$(PRODG)\bin
cplus_include_path=$(PRODG)\include\stlport-
4.5.3\stlport;$(PRODG)\include;$(DOLPHIN_INC);$(PRODG)
#cplus_include_path=$(PRODG)\bin\gcc3;$(PRODG)\include;$(DOLPHIN_INC)

linker_name=ngcld
rel_link_file="$(PRODG)\lib\plf keep.lst"
opt_assembler_name=ngcas
assembler_name=ngcas
linker_script=$(PRODG)\bin\ngc.ld
rel_linker_script=$(PRODG)\bin\plf.ld
stdlib=libsn.a libc.a libm.a libgcc.a libstl.a $(DOLPHIN_LIB)/si.a
$(DOLPHIN_LIB)/exi.a $(DOLPHIN_LIB)/base.a $(DOLPHIN_LIB)/os.a
$(DOLPHIN_LIB)/db.a $(DOLPHIN_LIB)/mtx.a $(DOLPHIN_LIB)/dvd.a
$(DOLPHIN_LIB)/vi.a $(DOLPHIN_LIB)/demo.a $(DOLPHIN_LIB)/pad.a
$(DOLPHIN_LIB)/ai.a $(DOLPHIN_LIB)/ar.a $(DOLPHIN_LIB)/ax.a
$(DOLPHIN_LIB)/dsp.a $(DOLPHIN_LIB)/card.a $(DOLPHIN_LIB)/hio.a
$(DOLPHIN_LIB)/mcc.a c:/gameOptixDDK/lib/HW2/amcExi2.a
c:/NPDP/GDEV/lib/OdemuExi2.a $(DOLPHIN_LIB)/gx.a $(DOLPHIN_LIB)/G2D.a
$(DOLPHIN_LIB)/texPalette.a $(DOLPHIN_LIB)/fileCache.a
$(DOLPHIN_LIB)/support.a $(DOLPHIN_LIB)/perf.a
```

In this setup, to change compilers, you only need to change the COMPILER_(X) variables at the start of the sn.ini.

Environment variables can also be used as expected. For example:

```
Assembler_path=%SN_PATH%\bin
```

libsn.a - SN Systems library

The archive libsn.a is the main repository for Gekko target-system code written by SN Systems. This includes DLL support and profiling code. You should always include libsn.a as the first item in your list of Gekko libs as this is where SN Systems will place other target-system code as and when necessary.

Obtaining the version for libsn.a

The __SN_Libsn_version global symbol allows you to obtain the library version at run time, e.g.

```
printf("libsn version = %d\n",__SN_Libsn_version[0]);
```

Chapter 3: The Visual Studio 6.0 integration

Overview of the Visual Studio 6.0 integration

This chapter describes how to integrate ProDG with Microsoft Visual Studio. Once you have successfully installed ProDG and selected the Visual Studio integration you will be able to create GameCube projects in Visual Studio, then build and debug them using ProDG Debugger.

The Visual Studio integration has the following main features:

- Builds your project using the ProDG compiler driver and linker
- Uses Visual Studio source browsing in your project
- Outputs compiler and linker errors/warnings in Visual Studio format so that double-clicking on a build error in the output window opens the source file on the appropriate line in the Visual Studio editor
- Enables ProDG debugger to be called directly from Visual Studio to debug the current project
- Imports and exports Visual Studio breakpoints at the start/end of a debug session in the ProDG debugger
- Ability to open source, shown in the ProDG debugger source pane, in the Visual Studio Editor to enable source file editing

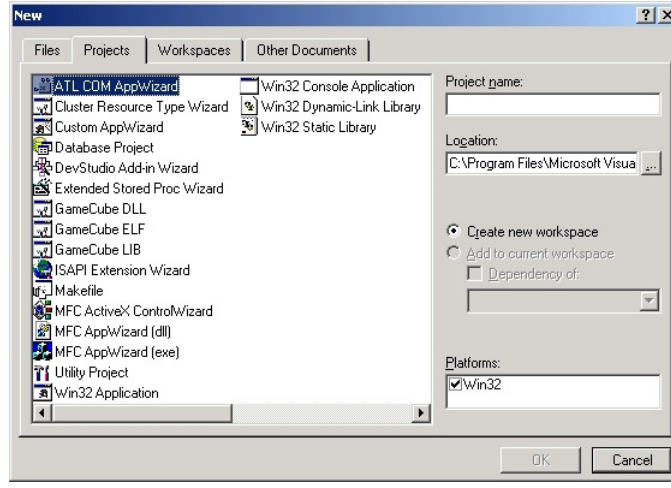
Setting up Visual Studio

When you start Visual Studio, immediately after doing the install, you will need to click **Customize** in the **Tools** menu, select the **Add-ins and Macro Files** tab, and in the dialog that appears select the **ProDG for GameCube Developer Studio Add-in** to enable it.

Creating a GameCube project in Visual Studio

Once you have completed the set up steps you can create a new GameCube project in Visual Studio by doing the following:

1. Click **New** in the **File** menu and click the **Projects** tab, and the following dialog is displayed:

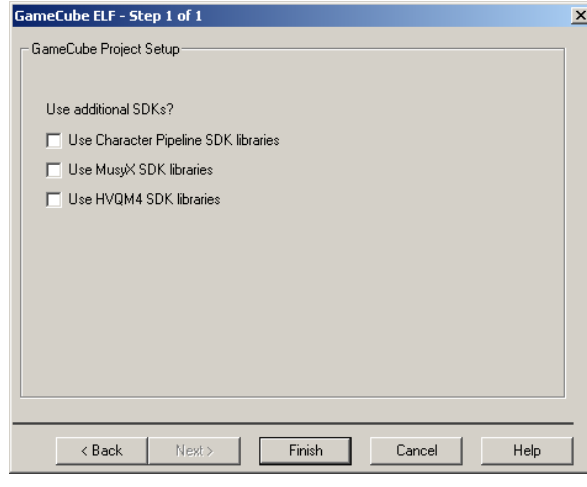


2. Select the GameCube AppWizard: **GameCube ELF**. (This step-through uses the **GameCube ELF** AppWizard as an example, but the same applies to the **GameCube LIB** or **GameCube DLL** AppWizards. Note that you will not be given the GameCube Project Setup dialog in step 5 in this case).

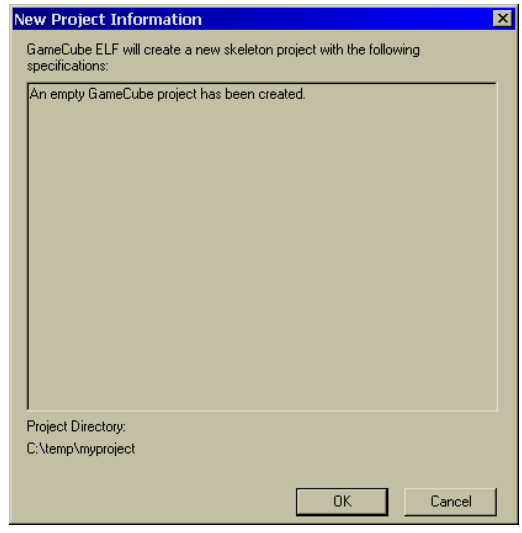
3. Enter the name of your new project in the **Project name** field.

4. In the **Location** field browse to the desired location for the new project file. Then click **OK**.

5. Select whether you wish to use the additional libraries provided by the **Character Pipeline SDK**, the **MusyX SDK** or the **HVQM4 SDK** from the dialog that appears. Any SDKs which are not installed on the local machine will not be selectable:



6. Click **OK** on the dialog that appears confirming that the new project has been successfully created and the new project is opened in Visual Studio:



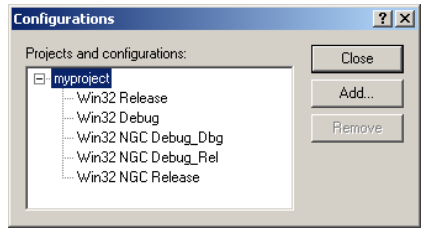
7. The standard SDK include paths are added to a new project by default. However If the project source files are in a different location to the Visual Studio project file, then Visual Studio will not be able to locate any non-SDK include files even if they are in the same directory as the source file. The solution to this is either:

   - (globally for all projects) to add the directory containing the project header files to the included files path in the **Directories** tab of the Options dialog (**Tools** menu); or

   - (for this project) click **Settings** in the **Project** menu, select the **C/C++** tab and with **Category: Preprocessor** selected set an additional include directory in the **Additional include directories** textbox.

The active project configuration

Each GameCube project generates debug and non-debug build configurations. Select **Configurations** from the **Build** menu option to display the projects and their configurations. A **GameCube ELF** has three possible build configurations, **Win32 NGC Debug_Dbg** (debug build using debug versions of Nintendo libraries), **Win32 NGC Debug_Rel Dbg** (debug build using release versions of Nintendo libraries), and **Win32 NGC Release** (release build using release versions of Nintendo libraries). The **Win32 Release** and **Win32 Debug** options are generated automatically and should be ignored.

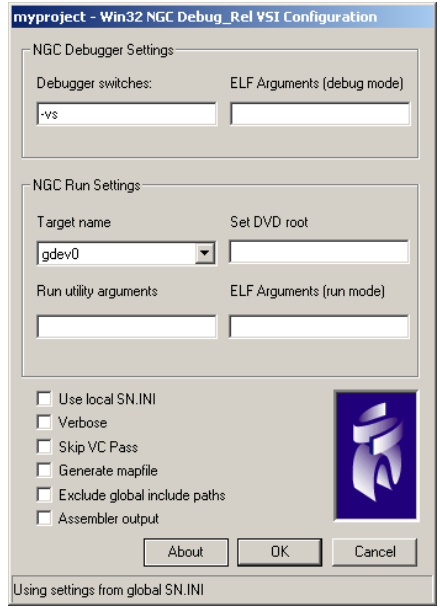


The active project configuration can be set by selecting **Set Active Configuration** from the **Build** menu.

Configuring the Visual Studio 6.0 integration

You can configure the way the Visual Studio integration behaves on a project by project basis, by creating a new project and then click the VSI Control Panel toolbar button:

The VSI Configuration dialog is displayed, similar to the following:



NGC Debugger Settings

Enables you to set the command-line switches and ELF command-line arguments passed to the ProDG Debugger program, ngcdbg.

Note: You should exercise great care in modifying the default values, as invalid switches may cause unexpected behavior in the ProDG Debugger.

NGCRUN Settings	Enables you to set the command-line switches and ELF command-line arguments passed to the ProDG command-line utility, ngcrun.
Use local SN.INI	By default, the Visual Studio integration takes its environment from the sn.ini file which is located in the directory pointed to by the SN_NGC_PATH= environment variable (see "sn.ini - SN Systems configuration file" on page 6). However, you can get it to use a local sn.ini file by checking this checkbox. If a local sn.ini file does not exist, you will be given the chance to create one.
Verbose	Enables verbose output during compiling.
Skip VC Pass	By default, C and C++ files are compiled twice in the Visual Studio integration: first using the Visual C compiler in order to do dependency checking, and then by the SN Systems compiler driver, ngccc. See "Using the ngccc compiler driver" on page 27 for more information). Check this option to skip the Visual C compiler pass and so speed up the compile.
Generate mapfile	Check this option to cause the linker to generate a mapfile.
Exclude global include paths	Check this option to avoid searching for an include file in the global include paths (as listed in **Tools** > **Options** > **Directories** > **Include files** view) but instead to search in the project directory only. In this way you can completely insulate your project from machine-dependent code in the global header files.
Assembler output	Check this option to cause the compiler to save an assembler listing.

Press **OK** to save your project settings in the sn.ini file, or **Cancel** to quit.

Building your GameCube project

With the Visual Studio integration you can build your GameCube project in the usual way in Visual Studio. This section tells you how to do it.

Adding your project files

The **GameCube ELF** new project is initialized with the ngc.ld linker script.

You now need to add your project source files to the new project using the **Add to Project** > **Files** command on the **Project** menu. The project source files needed by the project makefile (.c, .s etc.) must all be added to the project before carrying out the build.

Building your project

Building your project is carried out as usual, by invoking the **Clean**, **Compile** and **Build** (etc.) commands from the Visual Studio **Build** menu.

The following table lists the Visual C compiler switches and how they are translated into their GNU equivalents:

VC	GNU	Meaning
/D	-D	Defi